



American International University-Bangladesh (AIUB)

Faculty of Science & Technology (FST)

Department of Computer Science

Introduction to Data Science

Final-Term Project Report

Spring 2025-2026

Section: F

Group: 05

Project Title: Legal Document Type Discovery Using Unsupervised Clustering

Project Contributions

SL #	Student Name & ID	Contributions (mention every part of the project where you contributed)
1.	Name: TAMZID IQBAL ID: 22-46255-1	Data Collection, Text Preprocessing, Feature Representation using TF-IDF
2.	Name: ANIKA TABASSUM RIA ID: 22-49586-3	Dimensionality Reduction using UMAP, Unsupervised Clustering, Evaluation Metrics.
3.	Name: FAHIM FAISAL SHIFAT ID: 22-49603-3	Cluster Distribution Analysis, Evaluation Results, Cluster Interpretation.
4.	Name: MALIHA TUSNIM PRAPTI ID: 22-49917-3	Document Type Discovery, Visualization.

Legal Document Type Discovery Using Unsupervised Clustering

Tamzid Iqbal

22-46255-1@student.aiub.edu

American International University-Bangladesh
Bangladesh

Fahim Faisal Shifat

22-49603-3@student.aiub.edu

American International University-Bangladesh
Bangladesh

Anika Tabassum Ria

22-49586-3@student.aiub.edu

American International University-Bangladesh
Bangladesh

Maliha Tusnim Prapti

22-49917-3@student.aiub.edu

American International University-Bangladesh
Bangladesh

1 Methodology

The methodological framework implemented in this study is based on a sequential pipeline of document type discovery of legal texts by the use of unsupervised learning. The general workflow starts with collection of legal documents of different types and ends with text preprocessing, feature representation based on TF-IDF, dimensionality reduction using UMAP, clustering, evaluation and interpretation. As illustrated in Figure 1

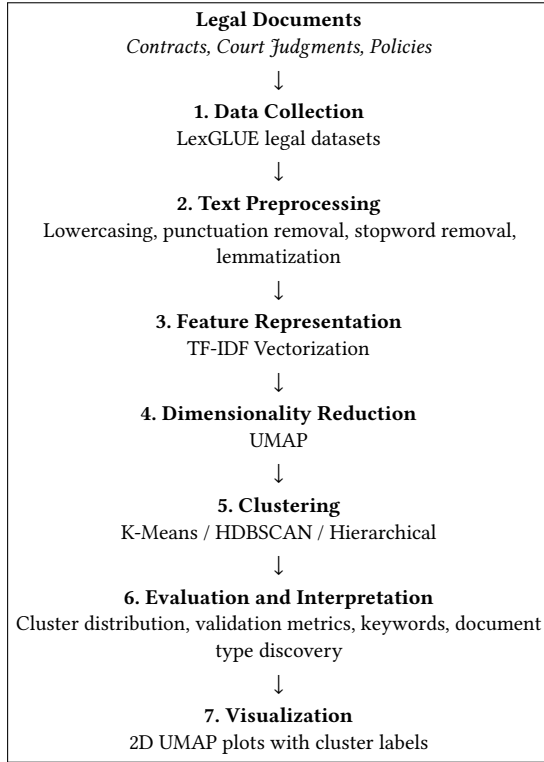


Figure 1: Proposed pipeline for legal document type discovery.

1.1 Data Collection

The experiments of this research are based on legal text datasets, which have been obtained via the LexGLUE benchmark, which is a publicly available collection of datasets, designed to support legal natural language processing tasks. The benchmark contains various legal corpora on various types of documents, e.g., contracts, court judgments, and texts related to policies.

In this work, three subsets of the LexGLUE dataset were considered: LEDGAR on contract documents, SCOTUS on court judgments, and UNFAIR-ToS on policy documents. Out of each subset, 1,000 documents were sampled out of the training split using the Hugging Face datasets API. Consequently, the resultant dataset is composed of 3,000 legal documents, with equal representation in the three classes.

The two main attributes of each document in the dataset are a category label that defines the type of document (Contract, Court Judgment, or Policy) and the textual content of the document. The processing and the merging of the datasets into a single structure was done via the usage of the library of panda and made sure that a consistency was made across the various sources.

The combined final dataset was saved in CSV format to be analysed further. The fact that the types of documents are distributed in a balanced manner allows reducing bias and guarantees more valid clustering results.

1.2 Text Preprocessing

The legal documents collected were preprocessed to turn the raw text into a more consistent and clean format that could be used in clustering. To ensure traceability between processed tokens and document categories, the following was done: Firstly, a unique document identifier was assigned to each document.

Simple text cleaning was then used on the textual data. All characters would be changed to lowercase so that they would be consistent throughout the dataset. HTML tags, numeric values, punctuation and non-alphabetic characters were eliminated. Also, additional whitespace was removed, to give a more structured and noiseless representation of the text. Following the cleaning stage, tokenization was performed to split each document into individual words. Let x represent a document. The tokenization process can be defined as:

$$T(x) = \{w_1, w_2, \dots, w_n\} \quad (1)$$

where $T(x)$ denotes the set of tokens extracted from document x .

After tokenization, standard English stopwords were removed as they do not carry significant semantic meaning for clustering. In addition, a set of domain-specific legal stopwords was excluded, including frequently occurring legal terms such as *shall*, *may*, *section*, *article*, *clause*, *hereby*, and *pursuant*. These terms are common in legal texts but provide limited discriminative value.

To further refine the token set, short and less informative tokens were filtered out. The resulting filtered token set is defined as:

$$T' = \{w \in T(x) \mid w \notin \text{StopWords}, \text{len}(w) \geq 3\} \quad (2)$$

Lemmatization was then implemented to normalize words into their base form and reduce the variations that have occurred due to the different grammatical structures. This action will be taken to ensure that words that are semantically similar will be considered as one feature during analysis.

The token dataset was cleaned in a preprocessing phase, which resulted in 3 attributes of the cleaned token dataset: document ID, category, and word. The processed tokens contain term related to the contract like that of set, debenture, company, legal, representative, and successor, indicating that vital legal characteristics were preserved post-processing.

1.3 Feature Representation using TF-IDF

The raw textual data was first preprocessed, then converted into a numerical format that could be used to cluster the data, using Term Frequency inverse Document Frequency (TF-IDF). TF-IDF is a statistical method that assesses the significance of a word in a document in comparison to the whole corpus. It gives increased weights to the terms that are frequent in a particular document but not frequent in other documents.

The TF-IDF of a term is given as:

$$TFIDF(t, d) = TF(t, d) \times \log \left(\frac{N}{DF(t)} \right) \quad (3)$$

where t represents a term and d represents a document. The term frequency $TF(t, d)$ measures how frequently a term appears in a document, while $DF(t)$ denotes the number of documents containing the term t , and N is the total number of documents in the dataset.

Once TF-IDF scores were computed, all terms with a weight of zero were dropped to reduce noise and increase the quality of features. The resultant information was then arranged as a Document-Term Matrix, where each row is a document and each column is a unique term.

Lastly, the Document-Term Matrix was transformed to a numerical matrix format to be used as input towards further dimensionality reduction and clustering. The resultant TF-IDF matrix was comprised of 2,999 documents and 31,060 distinct terms, containing some key legal and domain-specific words in the dataset.

1.4 Dimensionality Reduction using UMAP

The high-dimensional TF-IDF representations were reduced to a lower dimensional space by Uniform Manifold Approximation and Projection (UMAP). As in the TF-IDF matrix, there were many features (31,060 terms), dimensionality reduction was required to reduce the complexity of computations, and enhance the effectiveness of the clustering.

The choice of UMAP is because it can preserve both local and global structures of high-dimensional data and remain computationally efficient. UMAP offers a better representation to represent complex relationships in textual data, compared to the linear methods, namely Principal Component Analysis (PCA).

In this study, UMAP was configured with key parameters including the number of neighbors ($n_neighbors = 15$), minimum distance ($min_dist = 0.1$), and cosine distance as the similarity metric. These settings were chosen to capture the semantic relationships between legal documents while maintaining meaningful separation between clusters.

The dimensionality reduction process generated two components, *UMAP1* and *UMAP2*, representing each document in a two-dimensional space. These reduced features were subsequently used as input for clustering and visualization.

The resulting data set consisted of 2,999 documents, four attributes namely document ID, UMAP1, UMAP2 and category. To guarantee the process of dimensionality reduction can be reproducible, they used a fixed random seed.

1.5 Unsupervised Clustering

The clustering input was the reduced feature representations obtained with the help of UMAP. The two-dimensional feature space of UMAP1 and UMAP2 was used to run the clustering process. Three unsupervised clustering algorithms were used: K-Means, Hdbscan, and hierarchical clustering.

For K-Means clustering, the optimal number of clusters was determined using the elbow method. The within-cluster sum of squares was computed for different values of k ranging from 1 to 10. Based on the elbow curve, $k = 3$ was selected as the optimal number of clusters, corresponding to the three expected categories of legal documents. K-Means was then applied with three cluster centers and multiple random initializations to ensure stable clustering results.

HDBSCAN was also employed to identify density-based clusters in the UMAP space. Unlike K-Means, HDBSCAN does not require the number of clusters to be specified in advance. In this study, the minimum number of points parameter ($minPts$) was set to 15. This method is effective in detecting clusters of varying densities and identifying noise points within the dataset.

Besides that, Euclidean distance was computed on the UMAP coordinates, followed by hierarchical clustering. Ward's linkage method was used to minimize within-cluster variance during the merging process. This hierarchical clustering tree was further divided into three clusters to facilitate a comparison with the K-Means results.

The cluster assignments of all three methods were stored and used to further evaluate and interpret the clustering performance.

In order to further analyze the results of the clustering, cluster distributions were computed based on each algorithm by calculating the number of documents assigned to each cluster and computing the corresponding percentages.

1.6 Evaluation Metrics

The performance of clustering was measured in terms of both the internal and external validation measures, such as Silhouette Score, Normalized Mutual Information (NMI), and Adjusted Rand Index (ARI).

Silhouette Score was taken as an internal validation measure that is used to determine how the individual document fits within the cluster it belongs to as compared to other clusters. The higher the

values, the greater the cohesion and separation between clusters. As HDBSCAN labels certain documents as noise (cluster 0), the noise points were not counted in calculating the Silhouette Score to have a fair assessment.

Furthermore, NMI and ARI were also employed as the external measures of validation by comparing the predicted cluster labels with the actual document categories. NMI is used to measure the shared information between the clustering results and the ground truth labels as well as ARI is used to assess the similarity between the clustering assignments and the true labels adjusting the similarity to random chance. When both NMI and ARI have higher values, it means that they have more agreement with the real document types.

K-Means, hierarchical clustering, and HDBSCAN have computed these evaluation metrics to give a complete comparison of clustering performance.

2 Results and Discussion

In this section, the results achieved using the proposed clustering pipeline to analyze legal documents are presented. The results of the study are assessed using cluster distribution analysis, validation measures, and visualization as per the steps outlined in the methodology, such as feature extraction, dimensionality reduction and clustering. Moreover, the interpretation of clusters is conducted to receive the underlying document types in accordance with extracted textual characteristics.

2.1 Cluster Distribution Analysis

In order to study the distribution of documents within the clusters, the number of documents allocated to each cluster was calculated on all clustering algorithms.

In the context of K-Means clustering, the data set was broken down into three clusters of 1017, 1032 and 950 articles, respectively. The distribution is fairly uniform meaning that the algorithm divided the data equally, according to similarity.

Hierarchical clustering also gave out three clusters of 964, 1084 and 951 documents. Like K-Means, the distribution is relatively even, implying that there is a consistent tendency to group together.

Conversely, HDBSCAN yielded a number of clusters of different sizes as well as a large number of documents that were classified as noise (cluster 0), containing 1,020 documents. This shows that a good percentage of data is not found in areas of high density, which indicates the density-based nature of the algorithm.

In general, K-Means and hierarchical clustering yielded more balanced and interpretable cluster structures, whereas HDBSCAN identified more complex patterns in the data, by identifying variable-density clusters and isolating noise points.

2.2 Evaluation Results

Silhouette Score, Normalized Mutual Information (NMI) and Adjusted Rand Index (ARI) were calculated with each clustering algorithm to evaluate the quality of the clustering results.

The evaluation results are summarized in Table 1. Among the three algorithms, HDBSCAN achieved the highest Silhouette Score of 0.6989, indicating better cluster separation and cohesion compared to the other methods. In contrast, K-Means and hierarchical

clustering obtained Silhouette Scores of 0.5501 and 0.5404, respectively.

When comparing the external validation measures, K-Means demonstrated the highest NMI (0.8120) and ARI (0.8643) meaning that there was a significant agreement in the classification of the real document categories. Hierarchical clustering also fared well with a value of 0.7931 and 0.8440 of NMI and ARI respectively.

Conversely, HDBSCAN had lower values of NMI (0.3469) and ARI (0.3041), implying a weaker fit with the ground truth labels. This is mainly because of its clustering nature based on density, which recognizes complex structures and labels some documents as noise.

In general, the findings suggest that HDBSCAN is more effective in terms of internal cluster quality whereas K-Means and hierarchical clustering are more agreeable with the actual document categories based on external evaluation measures.

Table 1: Clustering Evaluation Results

Algorithm	Silhouette Score	NMI	ARI
K-Means	0.5501	0.8120	0.8643
HDBSCAN	0.6989	0.3469	0.3041
Hierarchical	0.5404	0.7931	0.8440

2.3 Cluster Interpretation

Cluster interpretation was done by taking out the most common words in each cluster. In each clustering algorithm the cleaned tokens were joined with the respective names of clusters using the document ID. Then, frequencies of words in each cluster were computed, and the 20 most common words in each cluster were chosen to reflect the most common words in that cluster.

For K-Means and hierarchical clustering, top keywords were extracted from all three clusters. These keywords help identify the semantic characteristics of each cluster and support the interpretation of document types. For example, clusters containing words such as *court*, *act*, *law*, *petitioner*, *federal*, *commission*, and *congress* indicate strong associations with court judgment documents. In the case of HDBSCAN, it was ensured that noise points marked as cluster 0 were eliminated and then top keywords were extracted. This was achieved due to noise points not being reflective of stable clusters and the resulting decrease in the interpretability of the keyword analysis. Once noise had been filtered out, the top keywords were mined out of the rest of the density-based clusters.

On the whole, the best keyword analysis offers an interpretative approach to understanding what each cluster means and helps to correlate the outcomes of the clustering with the types of legal documents (contracts, court convictions, and documents relating to policy).

2.4 Document Type Discovery

A mapping process was conducted on each cluster characteristics to determine the underlying document types represented by each cluster. In K-Means and hierarchical clustering, cluster labels were obtained by analyzing the top keywords that were obtained in each cluster. These keywords give semantic details about the content of

the documents in each cluster, which makes it possible to identify such categories as contracts, court judgments, and policies.

The clusters containing the terms court, act, law, petitioner, and federal were related to the court judgment documents, and the clusters containing the terms agreement, company, party, provision, borrower, and lender were related to the contract-related documents. Likewise, the word clusters that included service, content, copyright, account, and subscriber were perceived as documents that are policy-oriented.

In the case of HDBSCAN, the labeling of document types was not carried out in the same manner because it exhibits density based clustering behavior and has more than one cluster. All clusters were given a document type according to the prevailing actual category within the cluster. This was done to give a better interpretation of the clusters since HDBSCAN will give more smaller clusters and noise points.

Comparison of predicted document types and actual categories demonstrates that K-Means and hierarchical clustering had more consistent and interpretable mappings and HDBSCAN produced more fragmented assignments across document types.

2.5 Visualization

To visually analyze the clustering results, the UMAP-based two-dimensional representations of the dataset were plotted using the predicted document types as color labels. Figures ??, ??, and ?? illustrate the distribution of documents for K-Means, hierarchical clustering, and HDBSCAN, respectively.

The K-Means visualization depicts three different clusters, which relate to contract, court judgment, and policy documents. The clusters are fairly distant, with little overlap of categories, which implies that the dataset has been effectively partitioned.

In the same manner, hierarchical clustering results in well-defined clusters with a great distance between the three types of documents. The visual structure is in line with K-Means, which implies that it has a consistent behavior of clustering.

However, HDBSCAN visualization appears to have a more scattered design, and there is a distinct overlap between document types. This can be attributed to its density based nature which merely identifies clusters of different shapes and densities as opposed to the strict partitioning.

Overall, the visualizations confirm that K-Means and hierarchical clustering provide clearer separation of document types, while HDBSCAN captures more complex and less rigid cluster structures.

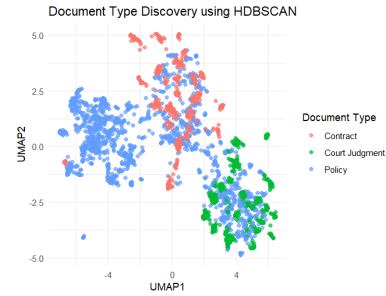


Figure 3: UMAP visualization using HDBSCAN.

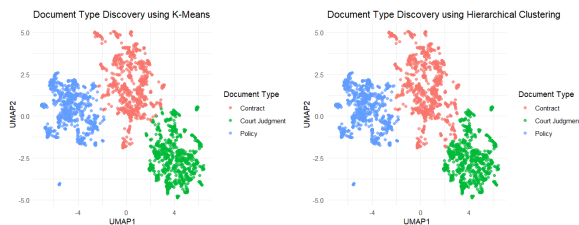


Figure 2: UMAP visualization using K-Means and Hierarchical Clustering.

Entire Project Code:

```
install.packages(c("readr", "dplyr", "stringr", "tidytext", "textstem"))
```

```
library(readr)
```

```
library(dplyr)
```

```
library(stringr)
```

```
library(tidytext)
```

```
library(textstem)
```

```
df <- read_csv( "legal_dataset_group_05_3k.csv", show_col_types = FALSE)
```

```
names(df)
```

```
head(df)
```

```
df <- df %>%
```

```
  mutate(doc_id = row_number())
```

```
df_clean <- df %>%
```

```
  mutate(
```

```
    Text = tolower(Text),
```

```
    Text = str_replace_all(Text, "<.*?>", " "),
```

```
    Text = str_replace_all(Text, "[0-9]+", " "),
```

```
    Text = str_replace_all(Text, "[[:punct:]]", " "),
```

```

Text = str_replace_all(Text, "[^a-z\\s]", " "),
Text = str_squish(Text)
)

data("stop_words")
custom_stopwords <- data.frame(
  word = c(
    "shall", "may", "must", "also",
    "section", "article", "clause", "subsection",
    "hereby", "herein", "thereof", "therein",
    "whereas", "pursuant", "including",
    "said", "upon", "within"
  )
)

tokens_clean <- df_clean %>%
  select(doc_id, Category, Text) %>%
  unnest_tokens(word, Text) %>%
  anti_join(stop_words, by = "word") %>%
  anti_join(custom_stopwords, by = "word") %>%
  mutate(word = lemmatize_words(word)) %>%
  filter(nchar(word) > 2)

head(tokens_clean)
dim(tokens_clean)

write_csv(tokens_clean, "legal_tokens_clean.csv")

```

```
library(dplyr)
library(tidytext)
library(tm)
library(readr)
```

```
word_counts <- tokens_clean %>%
  count(doc_id, word, sort = TRUE)
```

```
tfidf_data <- word_counts %>%
  bind_tf_idf(word, doc_id, n)
```

```
tfidf_data <- tfidf_data %>%
  filter(tf_idf > 0)
```

```
dtm <- tfidf_data %>%
  cast_dtm(doc_id, word, tf_idf)
```

```
matrix_data <- as.matrix(dtm)
```

```
dim(matrix_data)
head(tfidf_data)
```

```
write_csv(tfidf_data, "legal_tfidf.csv")
```

```
install.packages("uwot")
```



```
library(uwot)
```

```
set.seed(123)
```

```
matrix_data <- as.matrix(dtm)
```

```
umap_result <- uwot::umap(
```

```
  matrix_data,
```

```
  n_neighbors = 15,
```

```
  min_dist = 0.1,
```

```
  n_components = 2,
```

```
  metric = "cosine"
```

```
)
```

```
umap_df <- data.frame(
```

```
  doc_id = as.integer(rownames(matrix_data)),
```

```
  UMAP1 = umap_result[, 1],
```

```
  UMAP2 = umap_result[, 2]
```

```
)
```

```
category_info <- tokens_clean %>%
```

```
  distinct(doc_id, Category)
```

```
umap_df <- umap_df %>%
```

```
  left_join(category_info, by = "doc_id")
```

```
head(umap_df)
```

```
dim(umap_df)
```

```
write_csv( umap_df, "legal_umap_result.csv")
```

```
install.packages(c("ggplot2", "uwot", "dbscan"))
```

```
library(ggplot2)
```

```
library(uwot)
```

```
library(dbscan)
```

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

```
cluster_data <- umap_df[, c("UMAP1", "UMAP2")]
```

```
head(cluster_data)
```

```
dim(cluster_data)
```

```
set.seed(123)
```

```
wss <- numeric(10)
```

```
for (k in 1:10) {
```

```
  km <- kmeans(cluster_data, centers = k, nstart = 25)
```

```
  wss[k] <- km$tot.withinss
```

```
}
```

```
elbow_df <- data.frame(k = 1:10, wss = wss)
```

```
print(elbow_df)
```

```
plot(  
  elbow_df$k,  
  elbow_df$wss,  
  type = "b",  
  xlab = "Number of Clusters (K)",  
  ylab = "Within-Cluster Sum of Squares",  
  main = "Elbow Method for Optimal K"  
)
```

```
cluster_data <- umap_df[, c("UMAP1", "UMAP2")]
```

```
set.seed(123)  
kmeans_result <- kmeans(cluster_data, centers = 3, nstart = 25)  
umap_df$kmeans_cluster <- kmeans_result$cluster
```

```
library(dbSCAN)
```

```
hdbSCAN_result <- hdbSCAN(cluster_data, minPts = 15)
```

```
umap_df$hdbSCAN_cluster <- hdbSCAN_result$cluster
```

```
dist_matrix <- dist(cluster_data, method = "euclidean")
```

```
hc_result <- hclust(dist_matrix, method = "ward.D2")
```

```
hierarchical_result <- cutree(hc_result, k = 3)
```

```
umap_df$hierarchical_cluster <- hierarchical_result
```

```
head(umap_df)
```

```
table(umap_df$kmeans_cluster)
```

```
table(umap_df$hdbscan_cluster)
```

```
table(umap_df$hierarchical_cluster)
```

```
write.csv( umap_df, "legal_clustering_results.csv", row.names = FALSE)
```

```
table(umap_df$Category, umap_df$kmeans_cluster)
```

```
library(ggplot2)
```

```
ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = Category)) +
```

```
  geom_point(alpha = 0.7) +
```

```
  labs(
```

```
    title = "Actual Document Categories",
```

```
    x = "UMAP1",
```

```
    y = "UMAP2",
```

```

    color = "Category"
  ) +
  theme_minimal()

ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = as.factor(kmeans_cluster))) +
  geom_point(alpha = 0.7) +
  labs(
    title = "K-Means Clustering (K=3)",
    x = "UMAP1",
    y = "UMAP2",
    color = "Cluster"
  ) +
  theme_minimal()

ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = as.factor(hdbscan_cluster))) +
  geom_point(alpha = 0.7) +
  labs(
    title = "HDBSCAN Clustering",
    x = "UMAP1",
    y = "UMAP2",
    color = "Cluster"
  ) +
  theme_minimal()

ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = as.factor(hierarchical_cluster))) +
  geom_point(alpha = 0.7) +
  labs(

```

```
    title = "Hierarchical Clustering",  
    x = "UMAP1",  
    y = "UMAP2",  
    color = "Cluster"  
  ) +  
  theme_minimal()
```

```
plot(  
  hc_result,  
  labels = FALSE,  
  main = "Hierarchical Clustering Dendrogram",  
  xlab = "",  
  sub = ""  
)
```

```
kmeans_dist <- table(umap_df$kmeans_cluster)
```

```
print("K-Means Distribution:")  
print(kmeans_dist)
```

```
print("K-Means Percentage:")  
print(round(prop.table(kmeans_dist) * 100, 2))
```

```
hdbscan_dist <- table(umap_df$hdbscan_cluster)
```

```
print("HDBSCAN Distribution:")
```

```
print(hdbscan_dist)
```

```
print("HDBSCAN Percentage:")
```

```
print(round(prop.table(hdbscan_dist) * 100, 2))
```

```
hier_dist <- table(umap_df$hierarchical_cluster)
```

```
print("Hierarchical Distribution:")
```

```
print(hier_dist)
```

```
print("Hierarchical Percentage:")
```

```
print(round(prop.table(hier_dist) * 100, 2))
```

```
barplot(kmeans_dist,
```

```
  col = "skyblue",
```

```
  main = "K-Means Cluster Distribution",
```

```
  xlab = "Cluster",
```

```
  ylab = "Number of Documents")
```

```
barplot(hdbscan_dist,  
        col = "orange",  
        main = "HDBSCAN Cluster Distribution",  
        xlab = "Cluster",  
        ylab = "Number of Documents")
```

```
# Hierarchical Plot
```

```
barplot(hier_dist,  
        col = "lightgreen",  
        main = "Hierarchical Cluster Distribution",  
        xlab = "Cluster",  
        ylab = "Number of Documents")
```

```
install.packages("mclust")
```

```
library(cluster)
```

```
library(mclust)
```

```
cluster_data <- umap_df[, c("UMAP1", "UMAP2")]
```

```
dist_matrix <- dist(cluster_data)
```

```
silhouette_kmeans <- mean(silhouette(umap_df$kmeans_cluster, dist_matrix)[, 3])
```

```
silhouette_hier <- mean(silhouette(umap_df$hierarchical_cluster, dist_matrix)[, 3])
```



```

hdbscan_eval <- umap_df[umap_df$hdbscan_cluster != 0, ]
hdbscan_data <- hdbscan_eval[, c("UMAP1", "UMAP2")]
silhouette_hdbscan <- mean(
  silhouette(hdbscan_eval$hdbscan_cluster, dist(hdbscan_data))[, 3]
)

```

```

ari_kmeans <- adjustedRandIndex(umap_df$Category, umap_df$kmeans_cluster)
ari_hdbscan <- adjustedRandIndex(umap_df$Category, umap_df$hdbscan_cluster)
ari_hier <- adjustedRandIndex(umap_df$Category, umap_df$hierarchical_cluster)

```

```

calculate_nmi <- function(actual, predicted) {
  tab <- table(actual, predicted)
  n <- sum(tab)
  pxy <- tab / n
  px <- rowSums(tab) / n
  py <- colSums(tab) / n

  mi <- 0
  for (i in 1:nrow(pxy)) {
    for (j in 1:ncol(pxy)) {
      if (pxy[i, j] > 0) {
        mi <- mi + pxy[i, j] * log(pxy[i, j] / (px[i] * py[j]))
      }
    }
  }
}

```

```
hx <- -sum(px * log(px))
```

```
hy <- -sum(py * log(py))
```

```
mi / sqrt(hx * hy)
```

```
}
```

```
nmi_kmeans <- calculate_nmi(umap_df$Category, umap_df$kmeans_cluster)
```

```
nmi_hdbscan <- calculate_nmi(umap_df$Category, umap_df$hdbscan_cluster)
```

```
nmi_hier <- calculate_nmi(umap_df$Category, umap_df$hierarchical_cluster)
```

```
evaluation_results <- data.frame(
```

```
  Algorithm = c("K-Means", "HDBSCAN", "Hierarchical"),
```

```
  Silhouette_Score = c(silhouette_kmeans, silhouette_hdbscan, silhouette_hier),
```

```
  NMI = c(nmi_kmeans, nmi_hdbscan, nmi_hier),
```

```
  ARI = c(ari_kmeans, ari_hdbscan, ari_hier)
```

```
)
```

```
evaluation_results
```

```
library(dplyr)
```

```
tokens_kmeans <- merge(  
  tokens_clean,  
  umap_df[, c("doc_id", "kmeans_cluster")],  
  by = "doc_id"  
)
```

```
top_kmeans <- tokens_kmeans %>%  
  count(kmeans_cluster, word, sort = TRUE) %>%  
  group_by(kmeans_cluster) %>%  
  slice_max(n, n = 20) %>%  
  arrange(kmeans_cluster, desc(n))
```

```
tokens_hdbscan <- merge(  
  tokens_clean,  
  umap_df[, c("doc_id", "hdbscan_cluster")],  
  by = "doc_id"  
)
```

```
tokens_hdbscan <- tokens_hdbscan[tokens_hdbscan$hdbscan_cluster != 0, ]
```

```
top_hdbscan <- tokens_hdbscan %>%  
  count(hdbscan_cluster, word, sort = TRUE) %>%  
  group_by(hdbscan_cluster) %>%  
  slice_max(n, n = 20) %>%
```

```
arrange(hdbscan_cluster, desc(n))
```

```
tokens_hier <- merge(  
  tokens_clean,  
  umap_df[, c("doc_id", "hierarchical_cluster")],  
  by = "doc_id"  
)
```

```
top_hier <- tokens_hier %>%  
  count(hierarchical_cluster, word, sort = TRUE) %>%  
  group_by(hierarchical_cluster) %>%  
  slice_max(n, n = 20) %>%  
  arrange(hierarchical_cluster, desc(n))
```

```
print("K-Means Top 20 Words:")
```

```
print(top_kmeans)
```

```
print("HDBSCAN Top 20 Words (Noise removed):")
```

```
print(top_hdbscan)
```

```
print("Hierarchical Top 20 Words:")
```

```
print(top_hier)
```

```
kmeans_table <- top_kmeans %>%  
  group_by(kmeans_cluster) %>%  
  summarise(  
    top_words = paste(word, collapse = ", "),  
    .groups = "drop"  
  )
```

```
kmeans_table
```

```
kmeans_table$document_type <- NA
```

```
kmeans_table$document_type[kmeans_table$kmeans_cluster == 1] <- "Court Judgment"
```

```
kmeans_table$document_type[kmeans_table$kmeans_cluster == 2] <- "Contract"
```

```
kmeans_table$document_type[kmeans_table$kmeans_cluster == 3] <- "Policy"
```

```
kmeans_table
```

```
umap_df$kmeans_predicted_type <- kmeans_table$document_type[  
  match(umap_df$kmeans_cluster, kmeans_table$kmeans_cluster)  
]
```

```

table(
  Actual = umap_df$Category,
  Predicted = umap_df$kmeans_predicted_type
)

```

```

hier_table <- top_hier %>%
  group_by(hierarchical_cluster) %>%
  summarise(
    top_words = paste(word, collapse = ", "),
    .groups = "drop"
  )

```

```

hier_table

```

```

hier_table$document_type <- NA

```

```

hier_table$document_type[hier_table$hierarchical_cluster == 1] <- "Court Judgment"

```

```

hier_table$document_type[hier_table$hierarchical_cluster == 2] <- "Contract"

```

```

hier_table$document_type[hier_table$hierarchical_cluster == 3] <- "Policy"

```

```

hier_table

```

```

umap_df$hier_predicted_type <- hier_table$document_type[
  match(umap_df$hierarchical_cluster, hier_table$hierarchical_cluster)
]

```

```
table(  
  Actual = umap_df$Category,  
  Predicted = umap_df$hier_predicted_type  
)
```

```
hdbscan_type_table <- aggregate(  
  Category ~ hdbscan_cluster,  
  data = umap_df,  
  FUN = function(x) names(sort(table(x), decreasing = TRUE))[1]  
)
```

```
colnames(hdbscan_type_table) <- c("hdbscan_cluster", "document_type")
```

```
hdbscan_type_table
```

```
umap_df$hdbscan_predicted_type <- hdbscan_type_table$document_type[  
  match(umap_df$hdbscan_cluster, hdbscan_type_table$hdbscan_cluster)  
]
```

```
table(  
  Actual = umap_df$Category,  
  Predicted = umap_df$hdbscan_predicted_type  
)
```

```
library(ggplot2)
```

```
ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = kmeans_predicted_type)) +  
  geom_point(alpha = 0.7) +  
  labs(  
    title = "Document Type Discovery using K-Means",  
    x = "UMAP1",  
    y = "UMAP2",  
    color = "Document Type"  
  ) +  
  theme_minimal()
```

```
ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = hier_predicted_type)) +  
  geom_point(alpha = 0.7) +  
  labs(  
    title = "Document Type Discovery using Hierarchical Clustering",  
    x = "UMAP1",  
    y = "UMAP2",  
    color = "Document Type"  
  ) +  
  theme_minimal()
```

```
ggplot(umap_df, aes(x = UMAP1, y = UMAP2, color = hdbscan_predicted_type)) +  
  geom_point(alpha = 0.7) +  
  labs(  
    title = "Document Type Discovery using HDBSCAN",  
    x = "UMAP1",  
    y = "UMAP2",  
    color = "Document Type"  
  ) +  
  theme_minimal()
```



```
title = "Document Type Discovery using HDBSCAN",  
x = "UMAP1",  
y = "UMAP2",  
color = "Document Type"  
) +  
theme_minimal()
```